PHASE

5

Physical Memory Management

Find out which RAM really exists, and hand it out and take it back in 4 KiB page frames

Overall progress toward Turk-OS 1.0 (this phase in red) 28 % → 34 %

Duration	2 weeks
Difficulty	★★★☆☆ (3/5)
You need	Phase 4: working interrupts, timer, keyboard monitor
You will have	A parser for the Multiboot memory map, linker symbols that mark the kernel's start and end, a bitmap page-frame allocator with `pmm_alloc_frame`, `pmm_free_frame` and contiguous allocation, then a buddy allocator behind the same interface, a `mem` command with real numbers, and the first in-kernel test suite.

1 Why this phase

Every later subsystem needs memory: page tables (Phase 6), the kernel heap (Phase 7), process stacks (Phase 8), user address spaces (Phase 10), the buffer cache (Phase 12). All of it ultimately comes from one place, the **physical memory manager** (PMM), which knows which physical page frames are free and hands them out one at a time.

The PMM has to answer a harder question first: which memory can it hand out at all? Physical addresses are not a clean array of RAM. The first megabyte is full of BIOS data and video memory, the kernel image sits at 1 MiB, GRUB's structures and boot modules are scattered above it, and there are holes near the top of the 32-bit space for devices and firmware tables. Hand out the wrong frame and you will overwrite the kernel, or a device, and the crash will come much later somewhere unrelated. This phase is about getting that bookkeeping exactly right. It is also where you start writing **automated tests**, because a memory allocator that is "mostly right" is wrong.

2 Concepts you need

2.1 Address spaces and why the kernel needs an allocator

OSTEP chapter 13 introduces the **address space**, the memory a program believes it has, and explains why the OS must virtualise memory: isolation, protection, and the convenience of every program seeing memory starting at zero. You will build that illusion in Phases 6 and 10. Underneath it, the OS still manages real memory in fixed-size units called **page frames**. On x86 a frame is 4 KiB, and frame number n covers physical addresses $n \times 4096$ to $n \times 4096 + 4095$. Frame number and address convert with a shift by 12: `frame = addr >> 12`.

2.2 The Multiboot memory map

Because you set bit 1 of the Multiboot flags in Phase 2, GRUB passes a **memory map** (`mmap_addr`, `mmap_length` in the Multiboot information, valid when bit 6 of `flags` is set). It is the BIOS E820 map in a friendlier wrapper: a list of regions, each with a 64-bit base address, a 64-bit length and a type. Only type 1 is usable RAM; everything else (reserved, ACPI tables, bad memory) must never be allocated. Figure 1 shows a typical layout for a 128 MiB QEMU machine and what the PMM does with it.

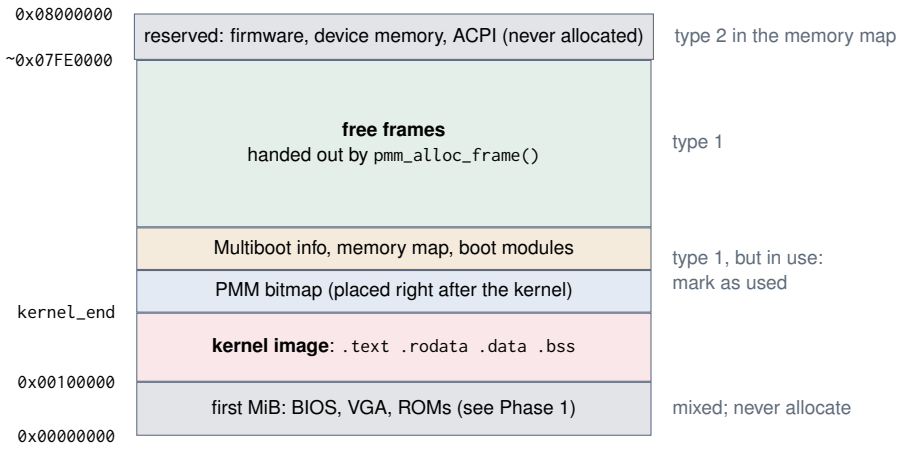


Figure 1: Physical memory of a 128 MiB QEMU machine (not to scale). Only frames in the green area are free.

2.3 Tracking free frames: bitmap or free list

There are two classic ways to track which frames are free, compared in MOS §3.2.3, OSDI §4.2 and OSTEP chapter 17. A **bitmap** uses one bit per frame. It is compact, and it is easy to find a run of free frames for a contiguous allocation, but finding a free frame means scanning. A **free list** chains free frames together, often by storing the "next" pointer inside each free frame itself, which gives O(1) allocation but makes contiguous runs hard to find. The table puts numbers on the choice.

	Bitmap	Free list (stack of frames)
Memory overhead	1 bit per 4 KiB frame: 128 KiB for all 4 GiB	none extra: the list lives in the free frames
Allocate one frame	scan for a 0 bit; fast with a "next free" hint	pop the head: O(1)
Free one frame	clear one bit: O(1)	push on the head: O(1)
Contiguous run of n frames	scan for n zero bits in a row	impractical
Detect double free	trivial: the bit is already 0	needs extra bookkeeping

Turk-OS starts with a bitmap: it makes bugs visible, supports the contiguous allocations that DMA needs in Phase 12, and reuses your Phase 0 bitmap code. You can add a free-list cache in front of it later if profiling says so.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	LOB	Ch. 10 *Page Frame Allocation*: how much memory is there, managing available memory	61–63	Tasks 5.1–5.3.
REQUIRED	OSTEP	Ch. 13 *The Abstraction: Address Spaces*	113–122	The goals that all of memory management serves.
REQUIRED	OSTEP	Ch. 17 *Free-Space Management*	161–178	Bitmaps, free lists, splitting, coalescing; you will reuse this in Phase 7.
RECOMMENDED	OSTEP	Ch. 14 *Interlude: Memory API*	123–134	The bugs allocator users make, so your tests look for them.

Priority	Book	What to read	Pages	Why now
RECOMMENDED	MOS	§3.1 *No Memory Abstraction*; §3.2 *Address Spaces* (esp. §3.2.3 *Managing Free Memory*)	182–194	Bitmaps versus linked lists with worked examples.
RECOMMENDED	OSDI	§4.1–4.2 *Basic Memory Management*, *Swapping* (bitmaps, linked lists)	374–383	The same ideas, with the MINIX perspective.
OPTIONAL	OSC	§9.1–9.2 *Background*; *Contiguous Memory Allocation*	349–360	Fragmentation, first/best/worst fit.
OPTIONAL	OSDI	§4.7.1 *Memory Layout* (MINIX process manager)	422–425	How MINIX lays out memory for processes.
REQUIRED	LKD	Ch. 12, *Pages*, *Zones*, *Getting Pages*	231–238	How Linux describes and hands out physical pages.
REQUIRED	LKP	Ch. 8, *Understanding and using the kernel page allocator (or BSA)*: freelists and scenarios	382–390	The buddy allocator of Task 5.7.

Real-system lens: how MINIX 3 does it

MINIX 3 as printed in OSDI does not use paging at all: the process manager allocates each process a contiguous chunk of physical memory from a list of *holes*. The hole list is explained on pp. 431 and 473–474 of OSDI; the file that implements it, `servers/pm/alloc.c`, is described there but not included in the printed listing. That is the linked-list scheme from MOS §3.2.3 used for whole processes. Turk-OS uses paging instead, so it only ever needs single 4 KiB frames, apart from the occasional contiguous DMA buffer. The comparison shows why paging won: no external fragmentation, and no need to move processes around.

Linux, for comparison. LKD LKP Linux manages physical memory with a **buddy allocator**: per zone, free lists of blocks of 2^{order} pages that are split on allocation and merged with their buddy on free. A bitmap is simpler, which is why you write it first, but the buddy allocator finds contiguous blocks without a scan (Task 5.7).

4 Build it

Task 5.1 Walk the memory map

Extend `multiboot.h` with the memory map entry and print the map at boot. The one trap: each entry starts with a `size` field that does *not* count itself, so the next entry is `size + 4` bytes further on.

kernel/mm/pmm.c (memory map walk)

```
struct mmap_entry {
    uint32_t size;          /* size of the rest of this entry, not counting this field */
    uint64_t base;
    uint64_t length;
    uint32_t type;          /* 1 = usable RAM */
} __attribute__((packed));

void print_memory_map(const struct multiboot_info *mbi)
{
    ASSERT(mbi->flags & (1 << 6));                  /* memory map present */
    uintptr_t p   = mbi->mmap_addr;
    uintptr_t end = mbi->mmap_addr + mbi->mmap_length;
    while (p < end) {
        const struct mmap_entry *e = (const struct mmap_entry *)p;
```

```
        kprintf("  %08lx%08lx  len %08lx%08lx  type %lu\n",
                (uint32_t)(e->base >> 32), (uint32_t)e->base,
                (uint32_t)(e->length >> 32), (uint32_t)e->length, e->type);
        p += e->size + sizeof(e->size);
    }
}
```

Verify: With `-m 128`, you see a usable region from 0 to about 640 KiB, reserved regions around the BIOS area, and a large usable region from 1 MiB to just under 128 MiB.

Task 5.2 Know where the kernel ends

The linker can define symbols at any point in the layout. Add `kernel_start` and `kernel_end` to `link.ld`; in C, declare them as arrays so you use their *address*, never their contents.

link.ld (changes)

```
SECTIONS
{
    . = 0x00100000;
    kernel_start = .;
    /* ... .text .rodata .data .bss as before ... */
    kernel_end = .;
}
```

kernel/mm/pmm.c

```
extern char kernel_start[], kernel_end[];     /* addresses only: never read them */
kprintf("kernel: %p - %p (%lu KiB)\n", kernel_start, kernel_end,
        (uint32_t)(kernel_end - kernel_start) / 1024);
```

Task 5.3 The bitmap allocator

Write `pmm_init(mbi)` in four steps. First, find the highest usable address to size the bitmap, and place the bitmap right after `kernel_end`, page-aligned. Second, mark *every* frame as used. Third, walk the memory map and free the frames of every type-1 region, rounding the start up and the end down to whole frames. Fourth, mark as used again everything below 1 MiB, the kernel image, the bitmap itself, the Multiboot information and memory map, and every boot module (`mods_addr`). Starting from "all used" means a mistake leaves memory unused instead of handing out memory that is in use.

include/pmm.h

```
#define PAGE_SIZE 4096

void      pmm_init(const struct multiboot_info *mbi);
uintptr_t pmm_alloc_frame(void);            /* returns a physical address, 0 if none free */
uintptr_t pmm_alloc_frames(size_t count);   /* physically contiguous run                   */
void      pmm_free_frame(uintptr_t paddr);
size_t    pmm_free_count(void);
size_t    pmm_total_count(void);
```

kernel/mm/pmm.c (allocation core)

```
static uint32_t *bitmap;          /* bit set = frame used */
static size_t    nframes, nfree, next_hint;

uintptr_t pmm_alloc_frame(void)
{
    for (size_t i = 0; i < nframes; i++) {
        size_t f = (next_hint + i) % nframes;
```

```
        if (!bitmap_test(bitmap, f)) {
            bitmap_set(bitmap, f);
            nfree--;
            next_hint = f + 1;
            return (uintptr_t)f * PAGE_SIZE;
        }
    }
    return 0;                    /* out of memory: frame 0 is never free, so 0 is safe */
}

void pmm_free_frame(uintptr_t paddr)
{
    ASSERT((paddr & (PAGE_SIZE - 1)) == 0);            /* must be frame-aligned */
    size_t f = paddr / PAGE_SIZE;
    ASSERT(f < nframes && bitmap_test(bitmap, f));     /* catches double frees  */
    bitmap_clear(bitmap, f);
    nfree++;
}
```

Steps three and four deserve their own function, because the memory map is input the kernel did not write. Firmware tables have bugs, and nothing stops an entry from claiming the kernel image is free RAM. In fact GRUB's map always does: the kernel sits inside the large usable region above 1 MiB. So treat every entry with suspicion. Skip an entry whose `size` is too small to hold the fields, an empty one, and one whose `base + length` wraps past 2^{64}. Clip the rest to the frames the bitmap covers, at most 4 GiB. Then reserve the kernel image, the bitmap, the Multiboot structures and the low 1 MiB again, whatever the map said about them. Because the function writes into a bitmap you pass in, a test can run it on a made-up map without touching the real allocator.

kernel/mm/pmm.c (applying the memory map)

```
struct range { uint64_t start, end; };              /* physical, [start, end) */

/* Free the usable frames of a memory map in bm, then reserve every range in keep[].
   pmm_init passes the real map; the tests pass a made-up one and a scratch bitmap. */
void pmm_apply_map(uint32_t *bm, size_t nframes, uintptr_t map, uint32_t map_len,
                   const struct range *keep, size_t nkeep)
{
    const uint64_t limit = (uint64_t)nframes * PAGE_SIZE;     /* at most 4 GiB */
    uintptr_t p = map;

    while (p + sizeof(struct mmap_entry) <= map + map_len) {
        const struct mmap_entry *e = (const struct mmap_entry *)p;
        if (e->size < sizeof(*e) - sizeof(e->size))
            break;                                  /* corrupt entry: stop the walk     */
        p += e->size + sizeof(e->size);
        uint64_t end = e->base + e->length;
        if (e->type != 1 || e->length == 0 || end < e->base)
            continue;                               /* not RAM, empty, or wraps: reject */
        if (end > limit)
            end = limit;                            /* clip to what the bitmap covers   */
        if (e->base >= end)
            continue;
        for (uint64_t f = (e->base + PAGE_SIZE - 1) / PAGE_SIZE; f < end / PAGE_SIZE; f++)
            bitmap_clear(bm, (size_t)f);
    }
    for (size_t i = 0; i < nkeep; i++)              /* kernel, bitmap, Multiboot, low 1 MiB */
        for (uint64_t f = keep[i].start / PAGE_SIZE;
             f < (keep[i].end + PAGE_SIZE - 1) / PAGE_SIZE && f < nframes; f++)
            bitmap_set(bm, (size_t)f);
}
```

`pmm_init` marks the whole bitmap used, calls `pmm_apply_map`, and then counts the clear bits to set `nfree`.

Verify: After `pmm_init`, print total and free frames. The free count should equal the usable RAM above the bitmap minus the Multiboot structures, give or take a few frames. `pmm_alloc_frame()` never returns an address inside the kernel.

Task 5.4 A tiny test framework

Write `KTEST(name)` and `KASSERT(cond)` macros and a `test` monitor command that runs every registered test and prints `PASS` or `FAIL` with the failing line. A simple way to register tests is an array of `{name, function}` pairs in `tests/tests.c`; a neater way is to put a pointer to each test in a dedicated linker section and walk it from `__start_ktests` to `__stop_ktests`. Phase 7 connects this to `make test` so it runs without you.

Task 5.5 Test the allocator hard

Write at least these tests. Each one should leave the free count exactly where it found it.

Test	What it checks
`alloc_free_one`	allocate a frame, check alignment and range, free it, count restored
`no_duplicates`	allocate 1000 frames into an array, check that no two are equal, free all
`exhaust`	allocate until 0 is returned, check the count reached zero, free everything, count restored
`never_kernel`	during `exhaust`, check that no frame overlaps `[kernel_start, kernel_end)` or the bitmap
`contiguous`	`pmm_alloc_frames(16)` returns 16 adjacent frames; free them one by one
`random_pattern`	10 000 random alloc/free steps against a shadow array; counts must match at every step
`bad_map`	run `pmm_apply_map` on a scratch bitmap with a made-up map: one usable entry covering the kernel image, one covering the Multiboot information, and one whose base plus length wraps past 2^{64}; afterwards no frame of the kernel image or the Multiboot data is free, and the wrapping entry freed nothing

Verify: All tests pass with `-m 16M`, `-m 128M`, `-m 512M` and `-m 3G`. The last one has a memory hole below 4 GiB, so it exercises the memory map handling.

Task 5.6 A real mem command

Update the monitor's `mem` command to print the memory map, the kernel's size, and total, used and free memory in MiB. You will watch these numbers constantly in later phases to spot leaks.

Task 5.7 A buddy allocator behind the same interface

The bitmap is easy to get right, but `pmm_alloc_frames(n)` has to scan for a free run, and freed runs never come back together. Production kernels use a **buddy allocator** instead (LKP ch. 8, LKD ch. 12): free memory is kept in lists by *order*, where a block of order k is 2^k contiguous, 2^k-aligned frames. To allocate order k, take a block from list k, or split a larger one in halves until it fits, putting the unused halves on the lower lists. To free a block, look at its **buddy**, the other half of the block it was split from, whose address differs in exactly one bit; if the buddy is free too, merge them and repeat one order higher.

Now that the bitmap version passes all its tests, replace its inside, not its interface: keep `pmm.h`, and make `pmm_alloc_frames(n)` round up to the next order. Keep one byte per frame for its state (free or not, and the order of the block it heads); the free lists can be the intrusive lists of Phase 0, threaded through the free frames themselves once paging maps them, or plain arrays of frame numbers until then.

kernel/mm/buddy.c (core)

```
#define MAX_ORDER 10                            /* blocks of up to 2^10 frames = 4 MiB */

static inline size_t buddy_of(size_t frame, unsigned order)
{
    return frame ^ ((size_t)1 << order);        /* flip the bit that separates the two halves */
}

void buddy_free(size_t frame, unsigned order)
{
    while (order < MAX_ORDER) {
        size_t b = buddy_of(frame, order);
        if (b >= nframes || !is_free_head(b, order))
            break;                              /* buddy busy, or split differently */
        list_remove_block(b, order);
        frame &= ~((size_t)1 << order);         /* the merged block starts at the lower half */
        order++;
    }
    list_add_block(frame, order);
}
```

Verify: The whole Task 5.5 suite passes unchanged on the buddy allocator. Freeing every frame you allocated leaves the free lists exactly as `pmm_init` built them, which you can check by printing the number of blocks per order (as Linux's `/proc/buddyinfo` does). Allocating 64 contiguous frames no longer takes time proportional to memory size.

5 Testing and debugging

Symptom	Likely cause	What to do
The memory map loop runs forever or prints garbage	Advanced by `sizeof(struct mmap_entry)` instead of `size + 4`, or struct not packed	Use the `size` field; check `sizeof` is 24.
Wrong totals on machines with more than 4 GiB	Base and length are 64-bit and were truncated	Keep them `uint64_t`; clamp regions to the 4 GiB you can address.
Crash long after boot, in code unrelated to the PMM	A frame inside the kernel, the bitmap, or the Multiboot data was handed out and overwritten	Run the `never_kernel` test; re-reserve those regions after freeing type-1 memory.
Off-by-one: last frame never allocated, or one frame past the end	Region end rounded the wrong way, or <= instead of <	Round the start up and the end down; write a test for a region ending exactly on a frame boundary.
Allocation gets slow as memory fills	Scanning from frame 0 every time	Keep a `next_hint`; later, scan 32 frames at a time by comparing whole words with `0xFFFFFFFF`.

Prepare for Phase 6

Right now physical address equals the address your code uses, so the PMM can read and write its bitmap directly. Once paging is on and the kernel moves to `0xC0000000`, that stops being true: the bitmap has a physical address and a different virtual one, and a frame returned by `pmm_alloc_frame` can't be touched until it is mapped. Keep all address arithmetic in the PMM explicit about which kind of address it is, for example by using a `typedef uintptr_t paddr_t` for physical addresses.

6 Milestone

Milestone: The kernel knows its memory

- ☐ The memory map is printed at boot, and `mem` shows totals that match QEMU's `-m`.
- ☐ `kernel_start` and `kernel_end` come from the linker script.
- ☐ `pmm_alloc_frame`, `pmm_free_frame` and `pmm_alloc_frames` work, with asserts for alignment and double frees.
- ☐ All seven PMM tests pass from 16 MiB to 3 GiB of RAM.
- ☐ Memory-map entries are validated: corrupt, empty and wrapping entries are rejected, the rest clipped, and the kernel and Multiboot data stay reserved (`bad_map`).
- ☐ The buddy allocator replaces the bitmap behind the same interface and passes the same tests; freeing everything restores the original free lists.
- ☐ The `test` command runs every registered test and reports the results.

Self-check questions

1. Why can't you assume that all memory from 1 MiB to `mem_upper` is free?
2. How big is a bitmap for 4 GiB of RAM with 4 KiB frames? For 128 MiB?
3. Why start by marking every frame as used, rather than every frame as free?
4. Why are `kernel_start` and `kernel_end` declared as arrays in C?
5. What is the difference between internal and external fragmentation? Which one can a page-frame allocator suffer from?
6. What would you need to change to make `pmm_alloc_frame` O(1)?

7 Going further

Compare your bitmap and buddy allocators with a benchmark of many small and large allocations, and read OSC §10.8.1 and OSTEP §17.4 for the textbook view of the buddy system. Split memory into zones, for example frames below 16 MiB for old DMA devices and the rest for everything else, as Linux does. Or add a small per-CPU cache of free frames in front of the bitmap, which is the standard way to make the common case O(1).

Next: Phase 6 Paging and the Higher-Half Kernel →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.